

Skillab

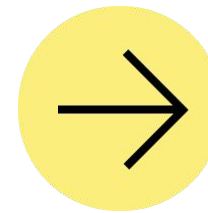
AI Agent Development

Richard Șerban Francu

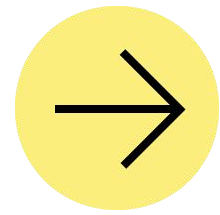
Senior AI Engineer @UCE Systems



Programa cursului



1. Noțiuni de bază — API-uri LLM și primul agent
2. Instrumente, prompturi și observabilitate
3. Procesarea și extragerea documentelor
4. Stocarea bazelor de date și RAG
5. **LangGraph — Stare și fluxuri de lucru**
6. Orchestrarea multi-agent
7. Optimizarea ML — Dincolo de apelurile LLM
8. Memorie și cache
9. Securitate și măsuri de protecție
10. Integrarea MCP
11. Inferența SageMaker
12. Training, Fine-Tuning și RLHF



LangGraph — Stare și fluxuri de lucru

Lecția 05

LangGraph — Stare și fluxuri de lucru

- Arhitectura bazată pe grafuri a LangGraph
- Definirea schemelor de stare (state schemas)
- Crearea nodurilor și muchiilor (nodes și edges)
- Construirea fluxurilor de lucru condiționale

Limitările LangChain

Ați întâmpinat deja aceste probleme:



Extracția eșuează → vrei retry

while loop manual, greu de urmărit



PDF → logică diferită față de CSV

if/else în cod, nu în flow



Vrei să oprești și să aștepți aprobare

Imposibil fără hack-uri



Debug: "unde a crăpat?"

Greu de vizualizat fluxul

Problema fundamentală

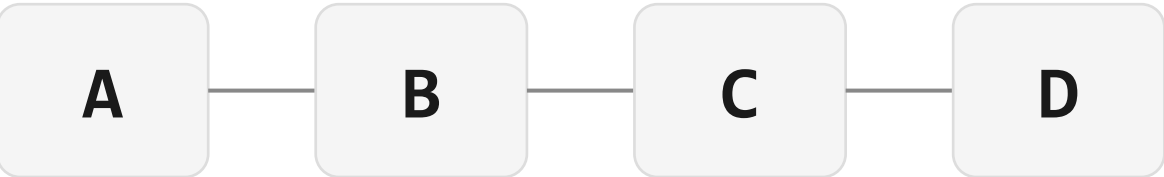
LangChain = **secvențial** (chain = lanț)

Lumea reală = **grafuri** (bucle, decizii, ramuri)

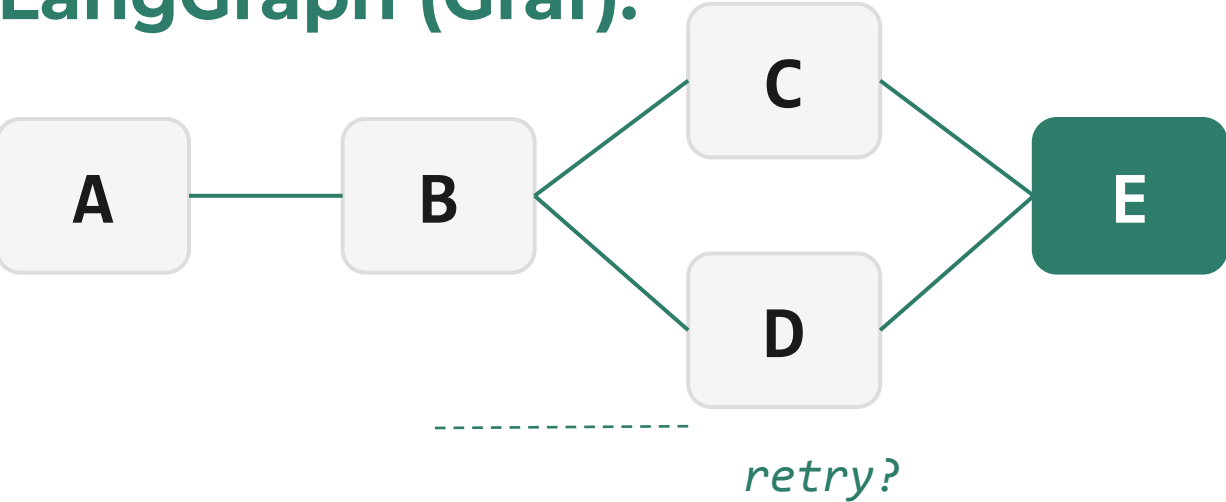
*"Un lanț merge doar înainte.
Un graf poate merge oriunde."*

LangGraph: Soluția

LangChain (Lanț):



LangGraph (Graf):



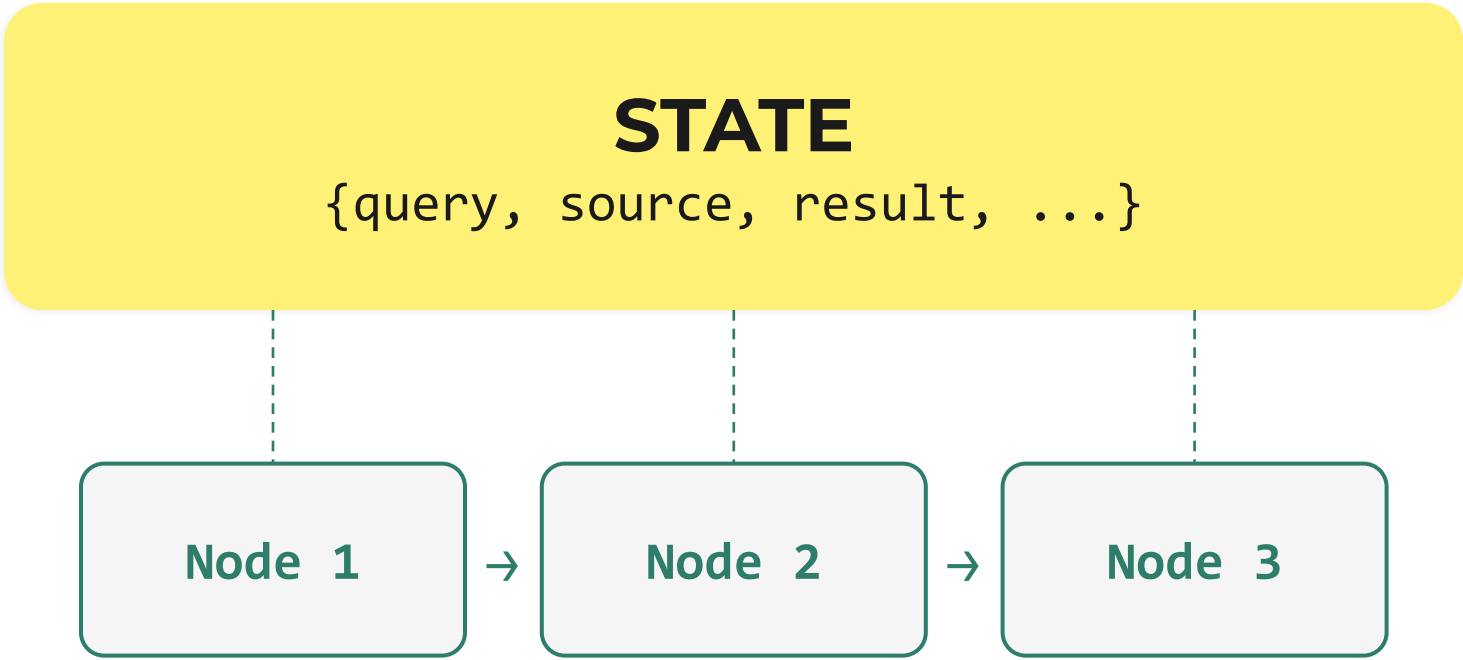
Capabilitate	LangChain	LangGraph
Execuție liniară	✓	✓
Loop-uri (retry)	✗	✓
Branching condiționat	✗	✓
State persistent	✗	✓
Vizualizare flow	✗	✓
Human-in-the-loop	✗	✓

→ Azi vom construi Data Reader-ul într-un graf LangGraph.

Ce este State-ul?

State = memoria workflow-ului

- Dictionar partajat între toate nodurile
- Fiecare nod citește și modifică state-ul
- Persistă pe toată durata execuției grafului



Diferența față de LangChain:

LangChain	LangGraph
Date pasate prin chain	State central, accesibil oriunde
Pierdere context între pași	Tot contextul păstrat
Greu de inspectat	State vizibil la orice moment

TypedDict — Schema pentru State



```
1  from typing import Optional, TypedDict
2
3
4  class DataReaderState(TypedDict):
5      # Input
6      query: str
7      max_retries: int
8
9      # Procesare
10     selected_source: str # "rag" | "csv" | "sql"
11     generated_query: str
12
13     # Tracking
14     retry_count: int
15     sources_tried: list[str]
16
17     # Output
18     result: Optional[dict]
19     error: Optional[str]
20
```

De ce TypedDict?



Type hints

IDE autocomplete



Validare

la compile time



Documentație

inline în cod

Input

query, max_retries

Procesare

selected_source,
generated_query

Tracking

retry_count,
sources_tried

Output

result, error

Channels și Reducers

 **Problema:** Două noduri modifică același câmp — cine câștigă?

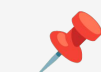
Channel = cum se stochează un câmp în state

Reducer = cum se combină valorile când mai multe noduri scriu

```
1 from operator import add
2 from typing import Annotated, TypedDict
3
4
5 class StateWithReducer(TypedDict):
6     # Ultimul câștigă (default, fără reducer)
7     current_source: str
8
9     # Lista se extinde (reducer = add → concatenare)
10    sources_tried: Annotated[list[str], add]
11
12    # Custom reducer: contor incremental
13    retry_count: Annotated[int, lambda a, b: a + b]
14
```

Reducers comuni:

Reducer	Comportament
(none)	Ultimul câștigă
add	Concatenează liste
lambda	Custom logic



Folosit pentru:

Valori simple, Istoric/logs,
Contoare și agregări

State-ul Data Reader-ului

Schema completă + flow-ul datelor

```
1  from typing import Annotated, Optional, TypedDict
2
3
4  class DataReaderState(TypedDict):
5      # Input
6      query: str
7      max_retries: int
8
9      # Decizie LLM
10     selected_source: str
11     generated_query: str
12
13     # Tracking (cu reducers)
14     retry_count: int
15     sources_tried: Annotated[list[str], add]
16
17     # Output
18     result: Optional[dict]
19     error: Optional[str]
20
```

Flow-ul state-ului:

Start

→

query: "vânzări Q1"
max_retries: 2

↓

Decide

→

selected_source: "sql"

↓

Execute

→

generated_query:
"SELECT * FROM sales..."

↓

Check

→

result: {...}
sources_tried: ["sql"]

Ce este un Nod?

Nod = funcție care transformă state-ul

- Primește state-ul curent
- Execută logică (LLM call, DB query, calcule)
- Returnează modificările pentru state



Reguli importante:

Regulă	De ce
Primește state: State	Acces la tot contextul
Returnează dict parțial	Doar câmpurile modificate
Fără side effects pe state	State-ul e immutable
0 singură responsabilitate	Ușor de testat și debug

"Un nod face UN lucru bine, nu mai multe lucruri prost."

Anatomia unui Nod



```
1 class DataReaderAgent:
2     # __init__, llm, tools, build_graph(), ...
3
4     def node_name(self, state: DataReaderState) -> dict:
5         # 1. Citește din state
6         query = state["query"]
7
8         # 2. Execută logica (folosind self.llm, self.tools, etc.)
9         result = self._do_something(query)
10
11        # 3. Returnează doar ce modifici
12        return {
13            "generated_query": result,
14            "retry_count": state["retry_count"] + 1,
15        }
16
```

1

Citește
din state

2

Execută
logica

3

Returnează
modificări

Tipuri de noduri:



LLM Node

Apelează modelul
`decide_source`



Tool Node

Execută acțiune
`execute_query`



Logic Node

Calcule, validări
`check_result`



Router Node

Decide direcția
(cu `edges`)

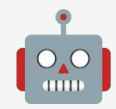
Nodurile Data Reader-ului

Cele 5 noduri ale grafului nostru:



parse_query

Analizează query-ul userului
Extrage intent, entități, constrângeri



decide_source

LLM alege sursa:
"rag" | "csv" | "sql"



generate_query

Creează query specific pentru sursă
SQL, embedding search, CSV filter



execute_query

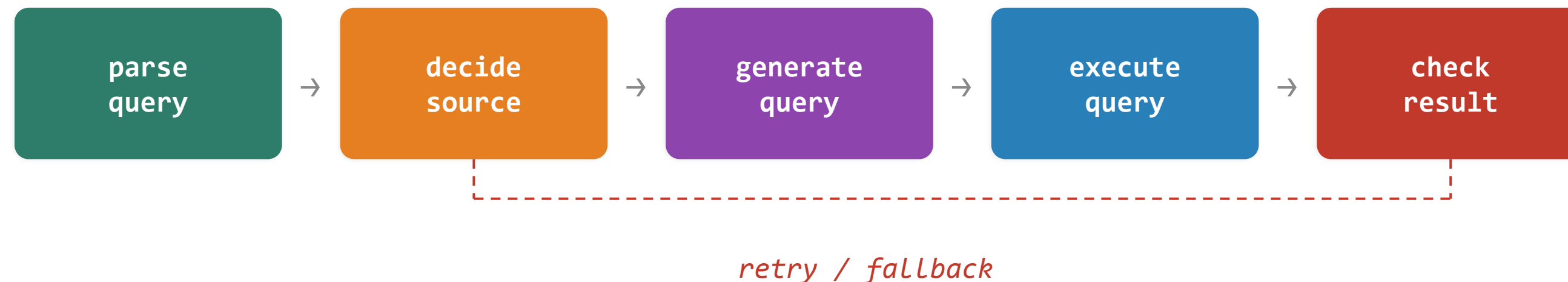
Execută pe sursa aleasă
Returnează date



check_result

Verifică: rezultat valid?
Gol? Eroare? → retry/fallback

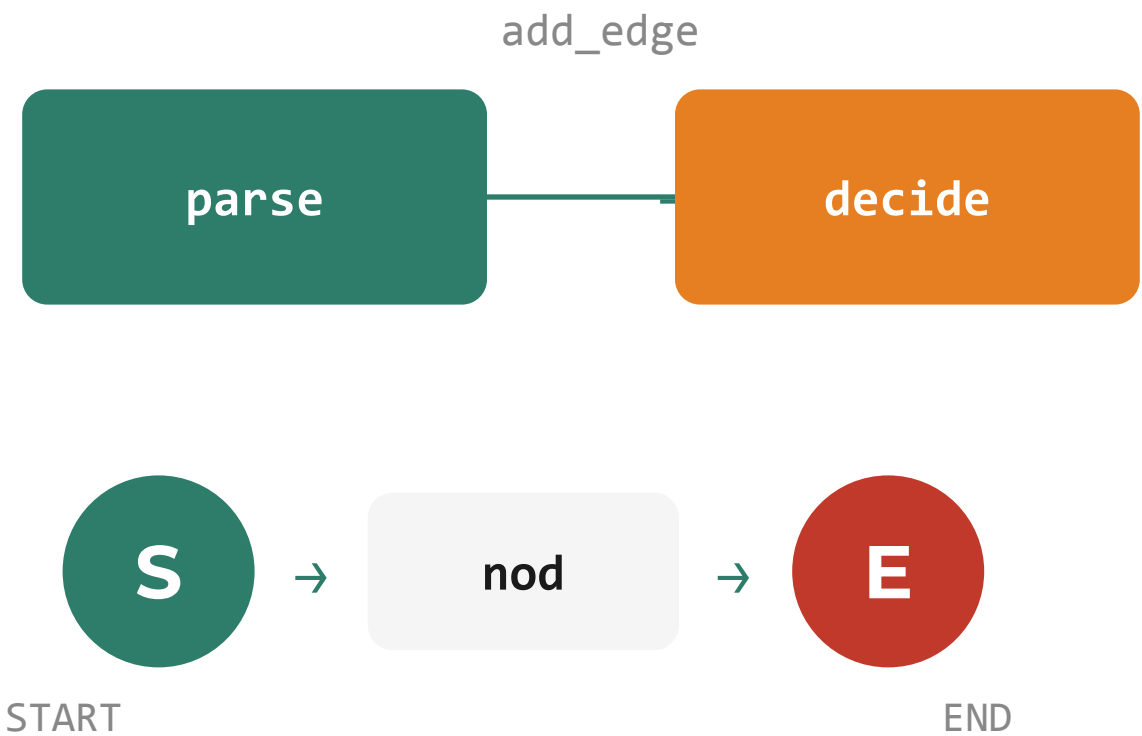
Flow:



Edges Simple

Edge = conexiune între noduri

```
1 class DataReaderAgent:
2     # __init__, parse_query, decide_source, ...
3
4     def build_graph(self) -> StateGraph:
5         graph = StateGraph(DataReaderState)
6
7         # Noduri (metode bound: self.parse_query, self.decide_source)
8         graph.add_node("parse", self.parse_query)
9         graph.add_node("decide", self.decide_source)
10
11        # Edges
12        graph.add_edge(START, "parse")      # intrarea în graf
13        graph.add_edge("parse", "decide")    # parse → decide (necondițional)
14
15        return graph
16
```



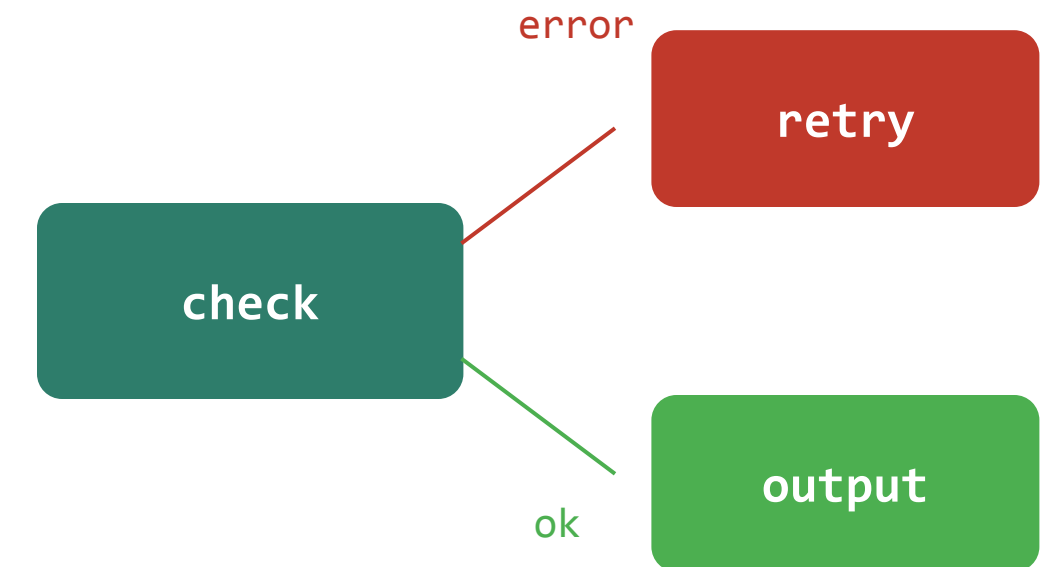
Edges speciale:

Edge	Semnificație
START → nod	Punctul de intrare în graf
nod → END	Terminare execuție graf
add_edge(a, b)	a → b mereu (necondițional)

Conditional Edges



```
1 def route_after_check(self, state: DataReaderState) -> str:
2     # Returnează NUMELE ramurii (nu nodul direct)
3     if state["error"]:
4         return "handle_error"
5     return "continue"
6
7 def _wire_check_routing(self, graph: StateGraph) -> None:
8     graph.add_conditional_edges(
9         "check",          # din ce nod
10        self.route_after_check, # funcția de routing
11        {
12            "handle_error": "retry",
13            "continue": "output",
14        },
15    )
16
```

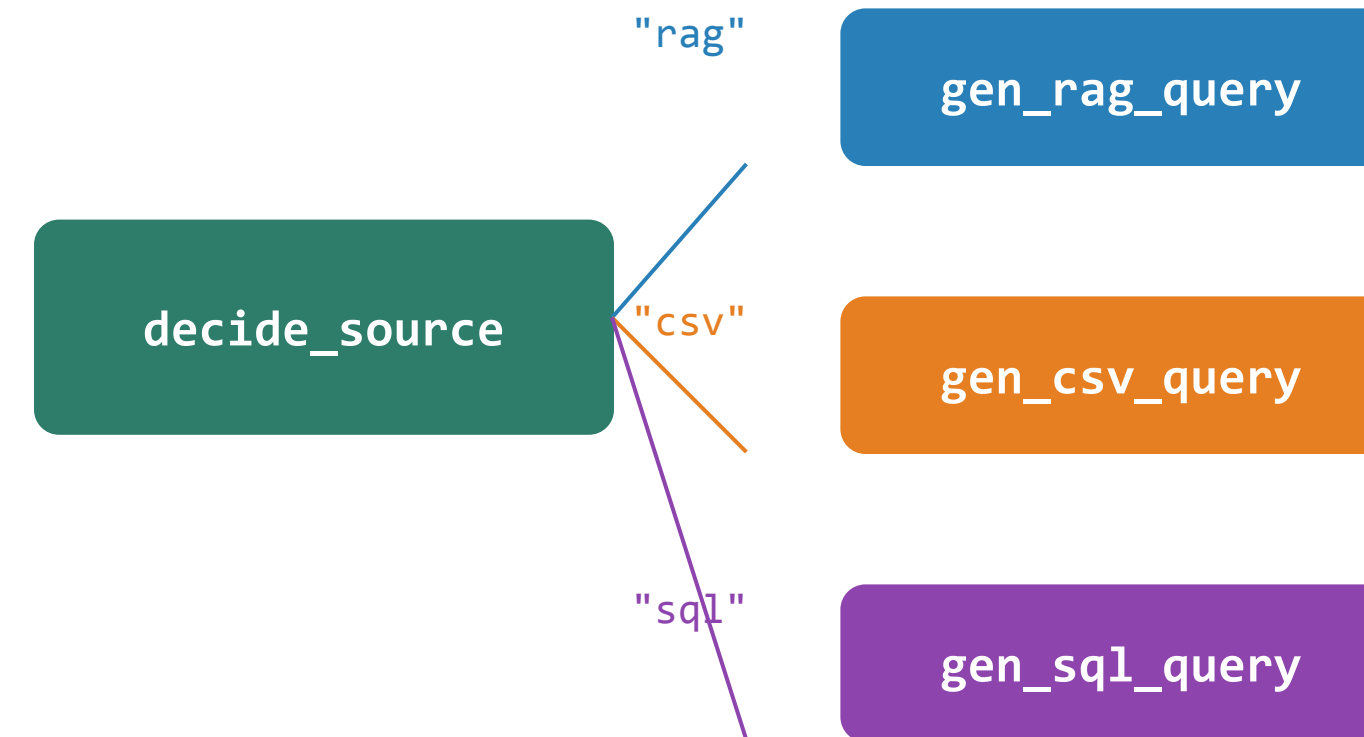


Regulă: Funcția de routing returnează un **str** (nume nod), nu nodul direct.

Routing pe Sursă

LLM-ul alege între RAG, CSV, SQL:

```
1 def route_to_source(self, state: DataReaderState) -> str:
2     return state["selected_source"] # "rag" | "csv" | "sql"
3
4     # În build_graph:
5     graph.add_conditional_edges(
6         "decide_source",
7         self.route_to_source,
8         {
9             "rag": "gen_rag_query",
10            "csv": "gen_csv_query",
11            "sql": "gen_sql_query",
12        },
13    )
14
```



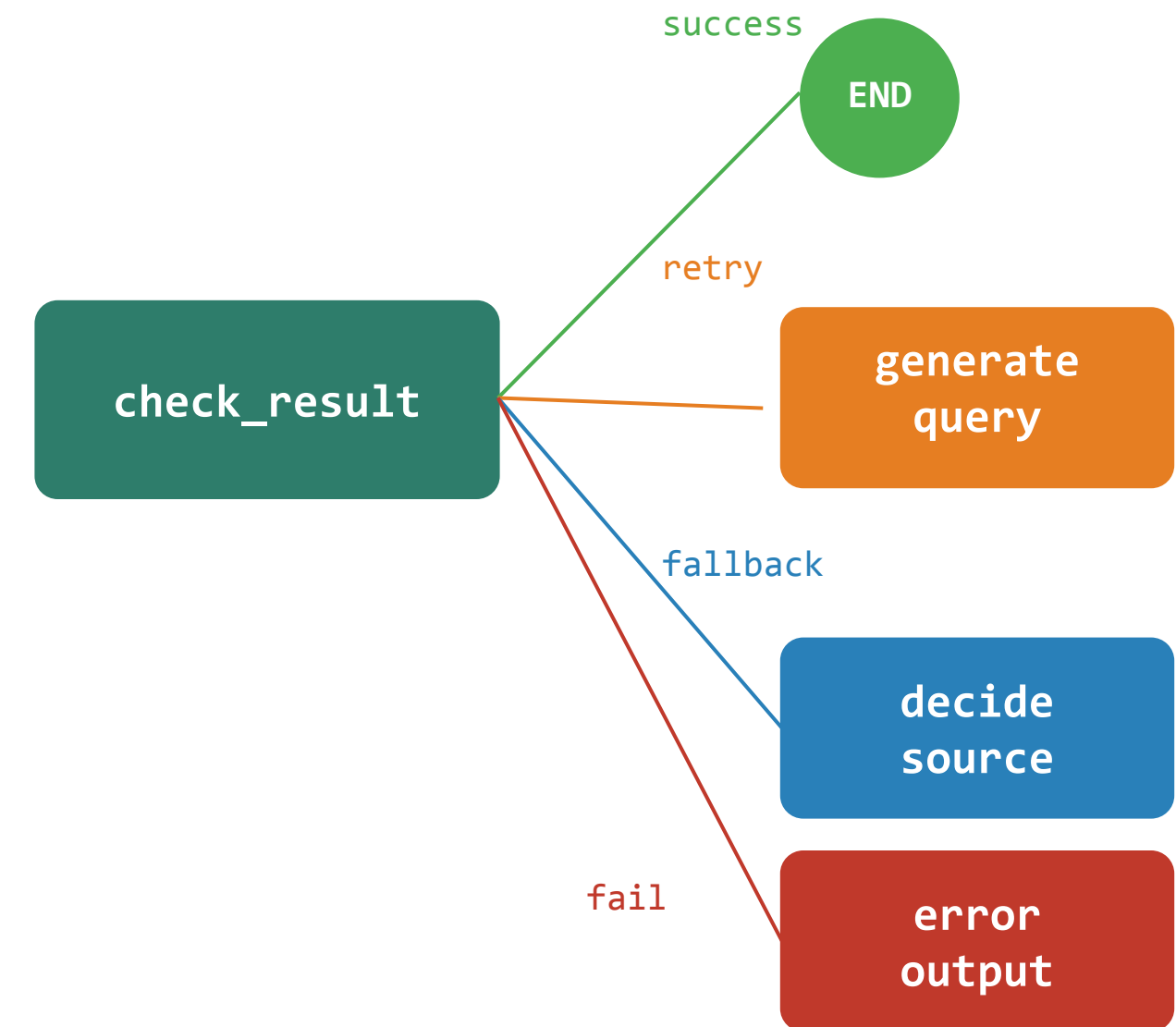
Pattern: Un singur nod decide → routing-ul distribuie pe ramuri specifice → fiecare ramură generează query-ul în formatul potrivit sursei.

Avantaj: Adăugarea unei surse noi = un nod + o ramură, fără a modifica restul.

Retry și Fallback

Logica complexă: succes / retry / fallback:

```
1 class DataReaderAgent:
2     AVAILABLE_SOURCES = {"rag", "csv", "sql"}
3
4     def route_after_check(self, state: DataReaderState) -> str:
5         # Succes
6         if state["result"] and not state["error"]:
7             return "success"
8
9         # Mai avem retry-uri pe sursa curentă?
10        if state["retry_count"] < state["max_retries"]:
11            return "retry"
12
13        # Mai există surse neîncercate?
14        if set(state["sources_tried"]) < self.AVAILABLE_SOURCES:
15            return "fallback"
16
17        return "fail"
18
```



Prioritate: success → retry (aceeași sursă) → fallback (altă sursă) → fail

Observație: Toată logica de decizie e în funcția de routing, nu în noduri. Nodurile rămân simple.

Asamblarea Grafului

Punem totul împreună:

```
1 class DataReaderAgent:
2     # __init__, noduri, routere ...
3
4     def build_graph(self):
5         # 1. Schema
6         graph = StateGraph(DataReaderState)
7
8         # 2. Noduri
9         graph.add_node("parse", self.parse_query)
10        graph.add_node("decide", self.decide_source)
11        graph.add_node("generate", self.generate_query)
12        graph.add_node("execute", self.execute_query)
13        graph.add_node("check", self.check_result)
14
15        # 3. Edge de start
16        graph.add_edge(START, "parse")
17
18        # 4. Edges simple
19        graph.add_edge("parse", "decide")
20        graph.add_edge("generate", "execute")
21        graph.add_edge("execute", "check")
22
23        # 5. Conditional edges
24        graph.add_conditional_edges("decide", self.route_to_source, ...)
25        graph.add_conditional_edges("check", self.route_after_check, ...)
26
27        # 6. Compile
28        return graph.compile()
29
```

Ordinea pașilor:

- 1 Schema de State**
TypedDict
- 2 Adaugă noduri**
add_node()
- 3 Edge START**
punctul de intrare
- 4 Edges simple**
add_edge()
- 5 Conditional edges**
routing logic
- 6 Compile**
graph.compile()



Ordinea contează!

Rularea Grafului

invoke() cu state inițial:

```
1 agent = DataReaderAgent(llm=..., tools=...)
2 app = agent.build_graph()
3
4 initial_state: DataReaderState = {
5     "query": "Vânzări din Q1 2024?",
6     "max_retries": 2,
7     "retry_count": 0,
8     "sources_tried": [],
9 }
10
11 result = app.invoke(initial_state)
12
13 result["selected_source"] # "sql"
14 result["result"]         # {"total": 150_000}
15 result["sources_tried"]  # ["sql"]
16
```

Ce se întâmplă:



→ **Graful returnează state-ul final complet** — toate câmpurile, inclusiv cele modificate pe parcurs.

Recap: Ce am învățat

LangGraph = grafuri pentru agenți complecși

Concept	Ce face	Exemplu Data Reader
State	Memoria workflow-ului	DataReaderState
TypedDict	Schema pentru state	Câmpuri tipizate + reducers
Node	Funcție care transformă state	parse, decide, execute
Edge	Conexiune între noduri	parse → decide
Conditional Edge	Routing bazat pe state	check → retry/fallback

Pattern-uri cheie:

 **Loop (retry)**
check → generate → execute → check

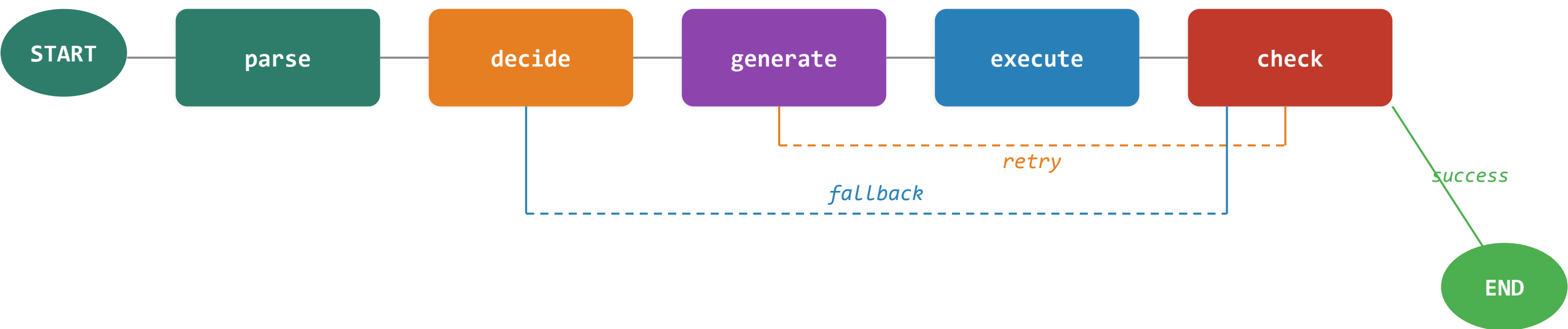
 **Branching**
decide → rag | csv | sql

 **Fallback**
check → decide (altă sursă)

De la LangChain la LangGraph: **Chains = liniar** → **Graphs = orice flow**

Arhitectura Data Reader

Graf complet cu toate nodurile și edge-urile:



Flows posibile:

Scenariu	Cale
✓ Succes direct	parse → decide → generate → execute → check → END
↺ Retry necesar	... → check → generate → execute → check → END
⚡ Fallback sursă	... → check → decide (altă sursă) → ... → END
✗ Eșec total	... → check → END (cu error)

Codul Complet

```
1 class DataReaderAgent:
2     # __init__, noduri (parse_query, decide_source, generate_query,
3     # execute_query, check_result), routere (route_to_source,
4     # route_after_check) ...
5
6     def build_graph(self):
7         graph = StateGraph(DataReaderState)
8
9         # Noduri
10        graph.add_node("parse", self.parse_query)
11        graph.add_node("decide", self.decide_source)
12        graph.add_node("generate", self.generate_query)
13        graph.add_node("execute", self.execute_query)
14        graph.add_node("check", self.check_result)
15
16        # Flow principal
17        graph.add_edge(START, "parse")
18        graph.add_edge("parse", "decide")
19        graph.add_edge("generate", "execute")
20        graph.add_edge("execute", "check")
21
22        # Routing pe sursă (toate ramurile converg la "generate")
23        graph.add_conditional_edges(
24            "decide",
25            self.route_to_source,
26            {"rag": "generate", "csv": "generate", "sql": "generate"},
27        )
28
29        # Routing după check
30        graph.add_conditional_edges(
31            "check",
32            self.route_after_check,
33            {
34                "success": END,
35                "retry": "generate",
36                "fallback": "decide",
37                "fail": END,
38            },
39        )
40
41        return graph.compile()
42
```

Schema

DataReaderState
TypedDict cu reducers

5 Noduri

parse → decide →
generate → execute → check

Flow liniar

START → parse → decide
+ generate → execute → check

Routing sursă

decide → rag/csv/sql
→ generate (același nod)

Retry/Fallback

check → retry/fallback/
success/fail

Demo: Rulare și Output

Exemplu 1: Succes direct

```
1 # Exemplu 1 – succes direct
2 result = app.invoke({
3     "query": "Clienți în martie?",
4     "max_retries": 2,
5 })
6 # selected_source: "sql"
7 # result: {"count": 1247}
8 # sources_tried: ["sql"]
9
10
```

Exemplu 2: Retry + Fallback

```
1 # Exemplu 2 – retry + fallback
2 result = app.invoke({
3     "query": "Contract X docs",
4     "max_retries": 1,
5 })
6 # selected_source: "rag" ← fallback după ce SQL a eșuat
7 # retry_count: 1
8 # sources_tried: ["sql", "rag"]
9 # result: {"documents": [...]}
10
```

Flow:



Observație: Graful a încercat SQL, a eșuat, a făcut **fallback automat pe RAG** — fără nicio intervenție manuală. Toată logica e în graf.

 **Homework: Extinde Data Reader cu NoSQL (MongoDB / DynamoDB) ca a 4-a sursă.**